BL-16 to BL-18: Attendance history, corrections, and modification history

Implements issues #16, #17, and #18 using FR-08, FR-10, FR-18, UC-07/UC-08, BL-01 D-09 and the BL-05 permission matrix.

History (BL-16)

Admin and Faculty open **Attendance history and corrections** from their dashboards. `/attendance/history/` lists student, subject, class/section, session date and period, current status and the original recording faculty. Recorder snapshots remain visible after recorder deletion. Inactive profiles are labelled; attendance is preserved after deactivation or enrollment removal.

Optional class/subject, student, inclusive start/end dates, period and status filters apply only to authorized records. Invalid filters never silently return the full list. The list is paginated, preserves filters across pages and shows an explicit empty state. Admin sees all records; Faculty see current assigned classes only. Students cannot access staff attendance or modification history.

Corrections (BL-17)

Open **Correct** beside a saved attendance record or from its modification history. `/attendance/records/<id>/correct/` accepts Present or Absent only. Faculty can correct their assigned classes until 23:59:59 IST on the last permitted calendar day. With the default two days, a 6 October session can be corrected through 8 October and is blocked from 9 October 00:00 IST. Every Admin correction requires a reason and is marked as an override, including corrections made within the Faculty window.

Admin can configure the institution-wide window at `/attendance/correction-window/`, from 0 through 30 whole calendar days. Zero allows only the session date. The reason is mandatory and the previous/new setting is audited. A database migration creates the singleton policy with a default of two days. Both missed-session creation and correction use the saved policy; changing the policy takes effect immediately.

The correction service reloads and locks the active actor, assignment and record before checking scope and changing the status. Cached roles, assignment ownership and statuses cannot grant stale access. Future sessions, invalid statuses and no-op changes are rejected. Student percentages, staff history, dashboard counts and shortage calculations query persisted records, so the corrected status appears on the next request without cached percentages.

Modification history (BL-18)

`/attendance/records/<id>/changes/` shows read-only corrections for the exact authorized record. Each entry includes editor identity, IST timestamp, previous/new status, reason and Admin override flag. Original recording identity remains distinct from correction editors. The pages remain available after re-login and enforce the same current assignment scope as the attendance list.

Status changes and their `attendance.correct` audit entries share one transaction. An audit failure rolls back the status change, and failed/no-op corrections create no completed-change entry. Concurrent corrections lock the record and record the actual preceding status in the audit chain. Existing append-only audit database triggers remain in force. Audit identities and target IDs are stored without foreign keys so entries survive account deletion.

All mutations require CSRF-protected POSTs. Staff history endpoints accept GET only; private responses use `Cache-Control: no-store`. An existing record outside scope produces the standard logged 403; a missing record produces 404.

Verification

```
python manage.py migrate
python manage.py test apps.attendance
python manage.py test
ruff check .
ruff format --check .
python manage.py check
python manage.py makemigrations --check --dry-run
```

Acceptance coverage	Tests
Authorized history, fields, filters, pagination, empty state, re-login	`apps.attendance.test_history`
Exact record audit scope, read-only history, recorder deletion, IST timestamps	`apps.attendance.test_history`
Correction persistence, Admin reason, unauthorized writes, live totals	`apps.attendance.test_corrections`, `apps.attendance.test_correction_views`
0/30-day policy bounds and database constraints, saved policy, reason/audit	`apps.attendance.test_corrections`, `apps.attendance.test_correction_views`
Last eligible IST instant, midnight expiry, active UTC timezone	`apps.attendance.test_corrections`, `apps.attendance.test_correction_views`
Record/audit rollback and concurrent correction audit chain	`apps.attendance.test_corrections`
Missing-session window uses the saved policy	`apps.attendance.test_corrections`, `apps.attendance.test_marking_window`

The concurrent tests use independent PostgreSQL connections. Disposable test cleanup temporarily permits truncating the audit table and restores its trigger in `finally`; production audit protection is unchanged.